Here are **detailed Week 6 notes** combining **Session VIII: Temporal Data II** and **Session IX: Model Understanding / Feature Importance & Selection**.

Week 6 — Working with Temporal Data + Model Understanding

Machine Learning and Advanced Analytics

1. Temporal vs non-temporal prediction problems

A **non-temporal problem** predicts a property of an unseen instance, but not necessarily a future quantity. For example, predicting whether a person has a certain fixed characteristic. The instance is usually a "thing", such as a customer, wine, product, or person.

A **temporal problem** predicts something in the future. Here, the instance is not just a "thing"; it is a **thing at a time point or time period**. For example, predicting whether customer Bob will churn next month means the instance is:

> *Bob at a particular reference date.*

So the same customer can appear multiple times in the dataset, because "Bob in January" and "Bob in February" are different temporal instances. The slides describe this as a `thing, time point/period` pair.

---

2. Why random hold-out is wrong for temporal problems

In normal machine learning, we often randomly split data into train and test sets. This works when rows are independent and there is no meaningful time ordering.

For temporal prediction, random hold-out can cause **future information leakage**. The model may indirectly learn information from future periods that would not be available at the prediction date.

Example: if we are predicting whether someone defaults next month, then at the reference date, we do not know next month's outcome yet. If random splitting puts some future-labelled rows into training and related rows into test, the model can learn from future behaviour. This gives an unrealistically high score.

The slides explain that randomized hold-out can inflate model performance because the training data may contain information that "in simulation world we cannot have known."

Exam wording

For temporal problems, randomized hold-out is invalid because it breaks the real-world prediction setting. In deployment, the model is trained only on past data and used to predict

future outcomes. Random splitting can allow information from future periods or shared temporal events to leak into training, producing overly optimistic performance estimates.

---

3. Temporal hold-out

A **temporal hold-out** evaluates the model using a realistic time split.

Instead of randomly splitting rows, we choose a **reference date**:

```
Past data → used for training
Future period after reference date → used for testing
```

For example:

```
Train: data up to day R
Test label: behaviour from R to R + β
```

In churn prediction, `R` could be the reference date and `β` could be the inactivity window. The model uses data before or up to `R`, then predicts whether the customer churns after `R`.

The important idea is:

> *The training set must only contain information that would have been available at the time of prediction.*

---

4. Repeated temporal hold-out

A single temporal hold-out only tests the model at one point in time. That may be misleading because customer behaviour, demand, fraud, or default patterns can change over time.

A **repeated temporal hold-out** evaluates the model across multiple reference dates.

Example:

```
Reference date 1 → train on past, test on next period
Reference date 2 → train on past, test on next period
Reference date 3 → train on past, test on next period
```

This gives a more reliable view of performance because the model is tested across different time periods.

The slides explain that using more than one reference point helps avoid assuming that future behaviour always follows the same pattern as one fixed month, such as assuming people default in every future month for the same reasons they did in July 2015.

Key benefit

Repeated temporal hold-out checks whether the model generalises over time, not just across randomly selected rows.

---

5. Why repeated randomized hold-out is also wrong

Repeated randomized hold-out, such as random cross-validation, repeats the same mistake multiple times. It still ignores time order.

Even if repeated random splits give stable scores, the scores may still be biased because future information can appear in training. So the issue is not only variance; it is the wrong evaluation design.

Exam wording

Repeated randomized hold-out is not suitable for temporal prediction because each random split can mix past and future rows. Repeating this process does not remove leakage; it only repeats the same unrealistic evaluation setup.

---

6. Reference dates and labelled temporal datasets

For temporal modelling, we usually construct labelled datasets around a **reference date**.

A labelled row usually contains:

```
Entity ID
Reference date
Features calculated before reference date
Label calculated after reference date
```

Example for customer churn:

```
customer_id
reference_date = R
features from transactions before R
label = churn between R and R + β
```

This structure prevents leakage because the features are historical and the label is future-facing.

---

7. Standardisation of temporal features

Temporal features created from aggregate windows often have different scales depending on the window size.

Example:

```
Spend in last 7 days
Spend in last 30 days
Spend in last 90 days
```

The 90-day spend will naturally be larger than the 7-day spend, even if customer behaviour is stable. Therefore, standardising temporal components independently can destroy meaningful comparisons.

Why independent standardisation is a problem

Suppose we standardise each window separately:

```
spend_7d standardised using 7d distribution
spend_30d standardised using 30d distribution
spend_90d standardised using 90d distribution
```

Then the model may not correctly understand whether the customer is increasing or decreasing over time, because each period has been transformed relative to its own distribution.

Correct idea

Temporal components should be standardised in a way that preserves their relationship. The model needs to compare windows meaningfully.

For example, if we have:

```
spend_last_30_days
spend_previous_30_days
```

we care about the difference or ratio between them. Standardisation should not remove that temporal signal.

---

8. Secondary temporal features

A **secondary temporal feature** is created from primary temporal aggregates.

Primary temporal features:

```
spend_last_30_days
spend_previous_30_days
visits_last_30_days
visits_previous_30_days
```

Secondary temporal features:

```
change_in_spend = spend_last_30_days - spend_previous_30_days
spend_ratio = spend_last_30_days / spend_previous_30_days
visit_change = visits_last_30_days - visits_previous_30_days
```

These features capture **trend**, **acceleration**, **growth**, **decline**, or **behavioural change**.

Why they matter

Raw aggregate windows tell us the level of behaviour. Secondary temporal features tell us how behaviour is changing.

For churn, a customer with high total spend may still be risky if their recent spend has dropped sharply.

---

9. Creating secondary temporal features in SQL

A basic SQL example:

```
SELECT
    customer_id,
    SUM(CASE WHEN day BETWEEN R - 30 AND R THEN sales_value ELSE 0 END) AS
spend_last_30,
    SUM(CASE WHEN day BETWEEN R - 60 AND R - 31 THEN sales_value ELSE 0 END) AS
spend_prev_30
FROM transactions
GROUP BY customer_id;
```

Then create secondary features:

```
SELECT
    customer_id,
    spend_last_30,
    spend_prev_30,
    spend_last_30 - spend_prev_30 AS spend_change,
    spend_last_30 / NULLIF(spend_prev_30, 0) AS spend_ratio
FROM customer_windows;
```

Use `NULLIF` to avoid division by zero.

---

10. SQL window functions for temporal data

SQL window functions are useful when we need values from previous rows, next rows, rolling counts, or cumulative totals.

Common functions:

```
LAG()
LEAD()
COUNT()
SUM()
AVG()
MAX()
MIN()
```

Common clauses:

```
PARTITION BY
ORDER BY
ROWS BETWEEN
```

Example:

```
SELECT
    customer_id,
    day,
    sales_value,
    LAG(sales_value) OVER (
        PARTITION BY customer_id
        ORDER BY day
    ) AS previous_sales_value
FROM transactions;
```

This finds the previous transaction value for each customer.

A rolling window example:

```
SELECT
    customer_id,
    day,
    SUM(sales_value) OVER (
        PARTITION BY customer_id
        ORDER BY day
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS rolling_7_transaction_sales
FROM transactions;
```

---

11. Time series vs event series

A **time series** has regular observations over time.

Example:

```
Daily sales:
Day 1: £100
Day 2: £120
Day 3: £90
```

An **event series** records irregular events.

Example:

```
Customer transactions:
Customer buys on day 3
Customer buys on day 10
Customer buys on day 45
```

Many business datasets are event series, especially transactions, clicks, complaints, logins, and purchases.

To use event series for temporal ML, we often convert them into time series-like features using aggregation windows.

Example:

```
Number of purchases in last 30 days
Total spend in last 30 days
Days since last purchase
Average gap between purchases
```

---

12. Two ways information can be included in time series analytics

There are two broad ways to include time information.

1. Direct time series modelling

The model directly predicts future values from previous values.

Example:

```
sales_t → sales_t+1
```

This is common in forecasting.

2. Feature-based temporal ML

We convert historical behaviour into features, then use a supervised ML model.

Example:

```
spend_last_30_days
visits_last_90_days
days_since_last_visit
average_inter_purchase_gap
```

Then predict:

```
churn_next_30_days
```

This is common in business analytics because it works well with customer-level, product-level, or store-level prediction tasks.

---

13. Baselines for temporal problems

Baselines are simple models used as a minimum comparison point. A machine learning model should beat a sensible baseline.

Important temporal baselines include:

Naive baseline

Assume the future equals the most recent observed value.

```
Tomorrow’s sales = today’s sales
```

Seasonal naive baseline

Assume the future equals the same period in the previous season.

```
This Monday’s demand = last Monday’s demand
```

Moving average baseline

Predict using the average of recent periods.

```
Next week’s demand = average demand over last 4 weeks
```

Business-rule baseline

Use an existing rule used by the organisation.

Example:

```
Flag customer as churn risk if they have not purchased for 60 days.
```

Majority-class baseline

For classification, always predict the most common class. This is useful but often weak for imbalanced problems.

Session IX — Model Understanding, Feature Importance & Selection

14. Association models vs causal models

Traditional machine learning usually builds **association models**.

It learns:

```
P(y | x, z)
```

This means:

> *Given observed inputs, what output do we expect?*

Causal inference asks a different question:

```
P(y | do(x = 1), z)
```

This means:

> *What would happen to the output if we intervened and changed x?*

The slides contrast machine learning as “what happens to my output if I observe my inputs?” and causal inference as “what happens to my output if I change my inputs?”

---

15. Variable importance vs causal importance

Variable importance means a feature helps the model predict the output.

Causal importance means changing the feature would change the output in the real world.

These are not the same.

Example:

A model may find that umbrellas are important for predicting rain. But umbrellas do not cause rain. They are associated with rain.

Exam wording

A variable can be predictively important without being causally important. Variable importance tells us which features help explain model predictions, whereas causal importance tells us which

features would change the outcome if intervened on.

---

16. Why feature importance is useful in business analytics

Feature importance can help with:

1. **Model refinement**
   Remove unnecessary features, reduce noise, reduce cost, and improve generalisation.
2. **Customer understanding**
   Identify characteristics of churners, defaulters, buyers, or high-value customers.
3. **Business action**
   Suggest candidate variables for possible intervention, although causal analysis is needed before acting.

The slides give a churn case study where variable importance is used to refine the predictive model, understand customers, and consider potential interventions.

---

17. Variable selection vs feature extraction

Variable selection keeps a subset of the original features.

Example:

```
Original features:
age, income, spend, visits, region

Selected features:
spend, visits, income
```

The features remain interpretable.

Feature extraction creates new transformed features from the original variables.

Example:

```
PCA component 1
PCA component 2
```

This may reduce dimensionality, but the new features can be harder to interpret.

Difference

Variable selection chooses original variables. Feature extraction creates new variables.

---

18. Univariate variable importance

Univariate methods evaluate each feature individually against the target.

Examples:

```
Correlation
Mutual information
Chi-square test
ANOVA
Variance filter
```

The slides describe univariate VIMs as ranking features based on how they individually affect the output feature. They are quick to compute and useful for identifying simple independent relationships.

Benefits

Fast, simple, easy to interpret.

Drawbacks

They ignore interactions. A feature may look unimportant alone but be useful when combined with another feature.

Example:

```
Feature A alone = weak
Feature B alone = weak
A + B together = strong
```

---

19. Model-based variable importance

Model-based importance comes from a fitted model.

Examples:

```
Decision tree feature importance
Random forest impurity importance
Linear regression coefficients
Logistic regression coefficients
```

Benefits

It uses the model's learned relationship, so it can capture more structure than univariate methods.

Drawbacks

Different models can give different importance scores. Some methods are biased towards variables with many possible splits or high cardinality.

---

20. Variable importance via model rebuilding

This method measures how performance changes when a feature or group of features is removed and the model is rebuilt.

Process:

```
Train model with all features
Record performance
Remove one feature or feature group
Retrain model
Compare performance
```

If performance drops a lot, the removed feature was important.

Benefits

Directly measures contribution to predictive performance.

Drawbacks

Computationally expensive because the model must be retrained many times.

---

21. Permutation importance

Permutation importance measures how much model performance drops when a feature is randomly shuffled.

Process:

```
Train model normally
Measure validation performance
Shuffle one feature column
Measure performance again
Importance = performance drop
```

If shuffling a feature damages performance, the model relied on that feature.

Benefits

Works with many models and is intuitive.

Drawbacks

Correlated features can cause misleading scores. If two variables contain similar information, shuffling one may not hurt much because the model can still use the other.

This is connected to **masking**.

---

22. Feature interactions

A feature interaction occurs when the effect of one feature depends on another feature.

Example:

```
Discount may increase purchase probability,
but only for customers who have bought before.
```

So the importance of discount depends on customer history.

Basic feature importance methods can miss interactions because they often evaluate features one at a time.

---

23. Masking effects

Masking happens when one feature hides the importance of another feature.

Example:

```
total_spend
average_spend
number_of_visits
```

These variables may be strongly related. If the model uses `total_spend`, then `number_of_visits` may appear less important, even though it contains useful information.

Exam wording

Masking occurs when correlated or interacting features share predictive information, causing one variable's importance score to appear artificially low because another feature already captures similar information.

---

24. Importance for sets of features

Sometimes individual variables look weak, but a group of related variables is important.

Example:

```
spend_last_7_days
spend_last_30_days
spend_last_90_days
```

Each feature may have modest importance, but together they represent customer spending behaviour.

So we may investigate feature groups:

```
spend features
visit frequency features
discount features
recency features
demographic features
```

This is useful when features are correlated or conceptually linked.

---

25. Best-k feature selection

Best-k feature selection chooses the top `k` features according to an importance score.

Example:

```
Select top 10 features by permutation importance.
```

Benefits

Simple and practical.

Drawbacks

The chosen number `k` is arbitrary unless tuned. It may remove features that are weak individually but useful in combination.

---

26. All-relevant feature selection

Best-k selection asks:

> *What is the best small subset of features?*

All-relevant selection asks:

> *Which features contain any useful information about the target?*

This is useful when the aim is not only prediction, but also understanding the problem domain.

For example, in business analytics, we may want to know all variables related to churn, not just the smallest set needed for prediction.

---

27. Boruta algorithm

Boruta is an all-relevant feature selection method.

The intuition:

1. Create random “shadow” versions of each feature.
2. Train a model, often a random forest.

3. Compare real feature importance against shadow feature importance.
4. Keep features that perform better than random shadow features.
5. Reject features that do not.
6. Repeat to make the decision more stable.

Boruta does not guarantee a globally optimal feature set. It is a heuristic method.

Exam wording

Boruta is an all-relevant feature selection algorithm. It compares real variables against randomised shadow variables and keeps variables that show stronger importance than the best random alternatives. It is useful for identifying all variables that may contain predictive signal, but it is heuristic rather than globally optimal.

---

28. Strong exam comparison table

Concept	Meaning	Why it matters
Temporal hold-out	Train on past, test on future	Simulates real deployment
Random hold-out	Random train/test split	Invalid for temporal prediction
Repeated temporal hold-out	Multiple time-based evaluations	Tests stability over time
Event series	Irregular events	Must be aggregated into temporal features
Time series	Regular time observations	Can be forecast directly
Secondary temporal feature	Feature created from temporal aggregates	Captures change/trend
Variable importance	Predictive usefulness	Helps model understanding
Causal importance	Effect of intervention	Helps decide actions
Masking	One feature hides another's importance	Important with correlated features
Boruta	All-relevant feature selection	Finds all useful variables, not just best-k

---

29. Likely exam-style answers

Why is randomized hold-out unsuitable for temporal prediction?

Randomized hold-out is unsuitable because it mixes past and future temporal instances. In real deployment, the model can only use information available before the prediction date. A random split may allow future labels, global events, or related future behaviour to influence training. This creates leakage and inflates model performance. Temporal hold-out is preferred because it trains on past data and evaluates on future data.

What is repeated temporal hold-out?

Repeated temporal hold-out evaluates a model across several reference dates. For each reference date, features are built using only historical data and labels are built from the following future period. This gives a more reliable estimate of performance because it tests whether the model works across different time periods rather than only at one point in time.

What are secondary temporal features?

Secondary temporal features are created by comparing or transforming primary temporal aggregates. For example, after calculating spend in the last 30 days and spend in the previous 30 days, we can create spend change or spend ratio. These features help the model capture trends, acceleration, decline, or behavioural change.

Difference between variable importance and causal importance

Variable importance tells us whether a feature helps predict the target. Causal importance tells us whether changing that feature would change the target. A feature can be predictively important without being causal. For example, umbrellas may predict rain, but umbrellas do not cause rain.

What is masking in feature importance?

Masking occurs when one feature hides the importance of another, often because the two features are correlated or contain overlapping information. If the model can use one variable to capture the same signal, the other variable may receive a low importance score even though it is genuinely useful.

What is Boruta?

Boruta is an all-relevant feature selection algorithm. It creates random shadow features, trains a model, and compares the importance of real variables against the shadow variables. Features that consistently perform better than random shadows are kept. It is useful for identifying all relevant predictors, but it is a heuristic rather than a guaranteed globally optimal solution.